हेलो! आपका स्वागत है **Unit 9: Objects** की complete study class में.

मैंने आपके workspace में, unit 9 objects folder के अंदर ये complete study files write कर दी हैं:

- Study Guide File: chapter9_objects.md
- Code Examples File: objects_examples.js
- Practice Exercises File: objects_practice.js

चलिए अब Objects को एकदम clear and simple Hinglish explanation के साथ समझना शुरू करते हैं.

PART 1: OBJECT BASICS

जब हमें एक ही entity (जैसे User Profile, Product details) से जुड़ी multiple details को group करके save करना हो, तब objects का use किया जाता है.

1. Object Kya Hai?

What is it? Object ek key-value paired data representation entity structure hai. Isme data ko store karne ke liye properties (keys) aur unki values ka set banaya jata hai.

Why is it needed? Real life objects properties (जैसे Car: brand, model, color) variable values ko clean aur group target formats mein manage karne ke liye.

Accessing properties

- **Keys:** Objects values identifiers.
- **Values:** Actual content.
- **Access options:** Dot notation (obj.key) or Bracket notation (obj["key"]).

Simple example

javascript

```
let studentProfile = {
   name: "Ravi",
   age: 20,
   course: "JS Pro"
};
console.log(studentProfile.name); // dot notation
console.log(studentProfile["course"]); // bracket notation
```

Output Ravi JS Pro

---

2. Dot Notation vs Bracket Notation

What is it?

- **Dot notation (object.property):** Properties access ka simple standard way.
- **Bracket notation (object["property"]):** String strings mapping key accesses.

When is Bracket Notation needed?

1. Jab key string space contain karti ho: user["full name"].
2. Jab query key parameters dynamic values target hold karta ho: user[activeKey].

Simple example

javascript

```
let user = {
   name: "Amit",
   "home address": "Delhi"
};
console.log(user.name);
console.log(user["home address"]); // Space keys require bracket notation
```

Output Amit Delhi

---

3. Add, Update & Delete Property

What is it? Properties structures modifications:

- **Add:** Assign values to new property key.
- **Update:** Assign new values to existing property keys.
- **Delete:** Clean property key properties using delete keyword.

Example

javascript

```
let laptop = {
   brand: "Dell"
```

};

laptop.ram = "16GB"; // add

laptop.brand = "HP"; // update

delete laptop.ram; // delete

console.log(laptop);

Output { brand: "HP" }

PART 2: NESTED STRUCTURES

1. Nested Objects

What is it? Object properties ke inside secondary objects values manage collections sets structure checks.

Example

javascript

```
let student = {
   name: "Aman",
   academic: {
      maths: 90,
      science: 85
   }
};
console.log(student.academic.maths); // 90
```

Output 90

---

2. Objects inside Arrays & Arrays inside Objects

- **Arrays inside Objects:** Object key data holds list arrays elements structure maps.
- **Objects inside Arrays:** Array elements indices hold individual objects list collections data points (Common JSON representation).

Example

javascript

```
// Array inside Object
let school = {
   classes: ["Class A", "Class B"]
};
console.log(school.classes[0]);
// Objects inside Array
let users = [
   { id: 1, name: "Rahul" },
   { id: 2, name: "Amit" }
];
console.log(users[1].name);
```

Output Class A Amit

PART 3: OBJECT HELPER METHODS

1. Object.keys(), Object.values(), Object.entries()

- **Object.keys(obj):** Returns array containing all keys strings names.
- **Object.values(obj):** Returns array containing all properties values.
- **Object.entries(obj):** Returns array of nested [key, value] pairs arrays.

Example

javascript

```
let item = { product: "Phone", price: 500 };
console.log(Object.keys(item));
console.log(Object.values(item));
console.log(Object.entries(item));
```

Output ["product", "price"] ["Phone", 500] [["product", "Phone"], ["price", 500]]

---

2. Object.assign()

What is it? Sare properties values source variables target objects variables clone merge.

Example

javascript

```
let source = { val: 99 };

let destination = { status: "Active" };

Object.assign(destination, source);

console.log(destination);
```

Output { status: "Active", val: 99 }

PART 4: DESTRUCTURING, SPREAD & SHALLOW COPY

1. Object Destructuring

What is it? Direct variables extract shortcut definitions object keys pull variables mapping formats.

Example

javascript

```
let user = { username: "amit_pro", email: "amit@mail.com" };

const { username, email } = user;

console.log(username);
```

Output amit_pro

---

2. Object Spread (...) & Shallow Copy

- **Spread operator (...):** Object properties ko unpack copy perform karke duplicates objects sets.
- **Shallow Copy:** Root primitive values copied, nested properties indices memory address references copy pointers coordinates shared pointers.

Example

javascript

```
let original = { name: "Ravi", location: { city: "Delhi" } };

let copy = { ...original }; // Shallow copy

copy.name = "Rahul"; // Primitive change stays in copy
```

copy.location.city = "Mumbai"; // Nested changes reflect in original as reference is shared

console.log("Original:", original);

console.log("Copy:", copy);

Output Original: { name: "Ravi", location: { city: "Mumbai" } } Copy: { name: "Rahul", location: { city: "Mumbai" } }

🧠 QUICK REVISION SUMMARY

- **Basics:** Key-value pairs store structure, brackets indexing helps accessing dynamic keys.
- **Add/Update/Delete:** obj.key = value sets, delete obj.key cleans.
- **Helper APIs:** Object.keys() yields key arrays, Object.values() yields value arrays, Object.entries() yields nested key-value lists.
- **Destructuring:** Quick variable extraction from keys.
- **Shallow Copy:** Root variables duplicates, nested objects reference keys share same memory addresses.

📝 IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **Object Literal:** Syntax defining key-value properties within curly brackets {}.
2. **Computed Property Keys:** Dynamic variable keys assignment notation inside brackets {[variable]: value}.
3. **Shallow Copy:** root properties values copying process preserving reference targets pointers to inner objects.
4. **Deep Copy:** Complete isolation copy duplicates all child nesting properties recursively.
5. **Property Shadowing (Objects):** Scope definitions nested key properties values evaluations layers.

💬 IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. Difference between Dot Notation and Bracket Notation?

Ans: Dot notation is cleaner but cannot access keys containing spaces or special characters. Bracket notation uses string evaluations to look up keys, enabling dynamic variable lookups and space characters keys mapping.

Q2. Difference between Shallow Copy and Deep Copy?

Ans: Shallow copy copies root primitive variables but shares inner nested object memory addresses. Deep copy duplicates all nested layers, separating object variables references completely.

Q3. Predict output:

javascript

```
let a = { code: 1 };

let b = a;

b.code = 2;

console.log(a.code);
```

Ans: 2. Objects are referenced-copied. Changing properties in one variable updates the original referenced memory block.

Q4. What does Object.keys() return?

Ans: Array of strings containing all keys names.

Q5. Predict output:

javascript

```
let profile = { id: 5 };

const { id } = profile;

console.log(id);
```

Ans: 5.

Q6. How to check if a key exists inside an object?

Ans: By using the "key" in object operator or checking if object.key !== undefined.

Q7. What does Object.entries() return?

Ans: Array of key-value pair arrays.

Q8. Predict output:

javascript

let item = { x: 5 };

delete item.x;

console.log(item.x);

Ans: undefined.

Q9. Predict output:

javascript

const obj = { val: 5 };

obj.val = 10;

console.log(obj.val);

Ans: 10. Assign with const isolates variable re-assignment pointers but child elements inside reference can be modified.

Q10. How do you merge objects using spread operators?

Ans: let merged = { ...obj1, ...obj2 };.

Q11. Predict output:

javascript

let name = "Ravi";

let profile = { name };

console.log(profile);

Ans: { name: "Ravi" }. (Property value shorthand).

Q12. Predict output:

javascript

let data = { list: [10, 20] };

console.log(data.list[1]);

Ans: 20.

Q13. Predict output:

javascript

let data = [{ val: 5 }];

console.log(data[0].val);

Ans: 5.

Q14. What does Object.assign() return?

Ans: The target object with updated merged properties.

Q15. Predict output:

javascript

```
let a = { name: "Pen" };

let b = { ...a };

b.name = "Pencil";

console.log(a.name);
```

Ans: "Pen". Spread creates a shallow copy, separating root property values.

Q16. Can we use numbers as keys in JavaScript objects?

Ans: Yes. The JavaScript engine coerces numeric keys to strings internally.

Q17. Predict output:

javascript

```
let obj = { 1: "One" };

console.log(obj["1"]);
```

Ans: "One".

Q18. How do you copy an object into a completely independent Deep Copy in JS?

Ans: By using JSON.parse(JSON.stringify(object)) or the modern native global method structuredClone(object).

Q19. Predict output:

javascript

```
let key = "brand";

let car = { [key]: "Toyota" }; // computed property keys

console.log(car.brand);
```

Ans: "Toyota".

Q20. Predict output:

javascript

console.log(typeof {});

Ans: "object".

💻 PRACTICE QUESTIONS

Question 1: Extract Zip code from nested object

- **Question:** Extract zip code from object: let user = { info: { address: { zip: 110001 } } }.
- **Solution:**

javascript

```
let user = { info: { address: { zip: 110001 } } };

let zipCode = user.info?.address?.zip;

console.log("Zip Code:", zipCode); // 110001
```

- **Explanation:** Traverses layers sequentially using optional chaining.

Question 2: Object Keys and Values loop

- **Question:** Print keys and values of { name: "Ravi", age: 20 } sequentially.
- **Solution:**

javascript

```
let profile = { name: "Ravi", age: 20 };

Object.keys(profile).forEach(key => {
   console.log(key + ": " + profile[key]);
});
```

- **Output:** name: Ravi age: 20
- **Explanation:** Loops through keys array, accesses properties using bracket notation.

Question 3: Destructure and Spread

- **Question:** Merge { a: 1 } with { b: 2 } and destructure the resulting variable values.
- **Solution:**

javascript

```
let part1 = { a: 1 };

let part2 = { b: 2 };
```

```
let combined = { ...part1, ...part2 };

const { a, b } = combined;

console.log(`a is ${a}, b is ${b}`); // a is 1, b is 2
```

- **Explanation:** Spread combines parameters, destructure extracts variables.

7:04 PM